SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I SHREE AISHWARYA.R(192521475) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Shree Aishwarya . R

Signature of the student:

QUESTION 1: Constraint-Based Problem Solving Using Backtracking Search

1.1 Problem Statement

A hospital must assign three doctors, $D1$, $D2$, and $D3$, to three shifts: Morning, Afternoon, and Night. The assignment must satisfy all the specified constraints.

The objective is to apply the Backtracking Search Algorithm to obtain a valid schedule by assigning values to variables, checking constraints, and reversing an assignment whenever a constraint is violated.

1.2 Definition of the Problem

Variables

Let the variables represent the shifts assigned to the doctors:

$$ X=\{D1,D2,D3\} $$

Domains

Each doctor can initially be assigned any of the following shifts:

$$ Domain(D_i)=\{M,A,N\} $$

Where:

- M = Morning
- A = Afternoon
- N = Night

Constraints

Constraint	Description
C1	Doctor D1 cannot work the Night shift.
C2	D2 must work before D3, meaning D2's shift must precede D3's shift.
C3	Only one doctor can occupy each shift.
C4	Doctor D3 cannot work the Morning shift.
C5	Every doctor must be assigned exactly one shift.

The shift order is defined as: $ M < A < N $

Therefore, the assignment must satisfy all five constraints simultaneously.

1.3 Backtracking Search Algorithm

Backtracking is a depth-first search technique used to solve constraint satisfaction problems. It assigns a value to a variable, checks whether the assignment is consistent with the constraints, and proceeds to the next variable if it is valid. If a violation occurs, the algorithm reverses the assignment and explores another possibility.

Algorithm

BACKTRACK(assignment):

If all doctors are assigned:

Return assignment

Select an unassigned doctor

For each available shift:

Assign the shift to the doctor

If all applicable constraints are satisfied:

Result = BACKTRACK(assignment)

If Result is successful:

Return Result

Remove the assignment

Return failure

1.4 Step-by-Step Execution

For clarity, the doctors are assigned in the order D1, D2, and D3.

1

Assign D1 to Morning

- C1: Satisfied because D1 is not assigned to Night.

- C3: Satisfied because Morning is unoccupied.
- C5: Satisfied because D1 has exactly one assignment.

Accepted

2

Assign D2 to Afternoon

- C2: D2 must work before D3. This can still be satisfied by assigning D3 to Night.
- C3: Satisfied because Afternoon is unoccupied.
- C5: Satisfied because D2 has exactly one assignment.

Accepted provisionally

3

Assign D3 to Night

- C2: Satisfied because Afternoon precedes Night.
- C3: Satisfied because Night is unoccupied.
- C4: Satisfied because D3 is not assigned to Morning.
- C5: Satisfied because D3 has exactly one assignment.

Accepted

All constraints are satisfied. Therefore, no backtracking is required for this assignment sequence.

1.5 Constraint Verification

Constraint	Final assignment	Status
C1: D1 cannot work Night	D1 → Morning	Satisfied
C2: D2 works before D3	D2 → Afternoon; D3 → Night	Satisfied
C3: One doctor per shift	All shifts are distinct	Satisfied
C4: D3 cannot work Morning	D3 → Night	Satisfied
C5: Every doctor has one shift	All doctors assigned once	Satisfied

1.6 Final Valid Schedule

Hospital Doctor Shift Allocation

Shift	Assigned Doctor
Morning	D1
Afternoon	D2
Night	D3

1.7 Advantages of Backtracking Search

1. Constraint satisfaction: It systematically checks whether assignments satisfy all requirements.
2. Reduced unnecessary exploration: Invalid partial assignments can be rejected before completing the entire schedule.
3. Completeness: For a finite search space, it can find a solution if one exists, provided the search explores the available alternatives.
4. Flexibility: It can be adapted to larger scheduling problems involving more doctors, shifts, and restrictions.

Conclusion

Backtracking Search successfully solves the hospital scheduling problem by assigning D1 to Morning, D2 to Afternoon, and D3 to Night. The final assignment satisfies every stated constraint and provides a valid schedule without requiring backtracking. This demonstrates the practical application of constraint satisfaction techniques in automated scheduling.

QUESTION 2: Grid Navigation Using Breadth-First Search (BFS)

2.1 Problem Statement

A robot navigates a (5 \times 5) grid from Start (S) to Goal (G), avoiding obstacles. It can move Up, Down, Left, or Right, and each move costs 1. The objective is to find the shortest path using BFS.

Assumption: Since exact obstacle coordinates are not provided, this answer uses an illustrative grid. Obstacles are located at ((2,2)), ((2,3)), ((4,3)), and ((4,4)). The start is ((1,1)), and the goal is ((5,5)).

2.2 Breadth-First Search (BFS)

BFS is an uninformed search algorithm that explores nodes level by level using a FIFO (First In, First Out) queue. When all moves have equal cost, BFS finds a path with the minimum number of moves.

Algorithm

BFS(Start, Goal):

Create an empty FIFO queue

Insert Start and mark it visited

While the queue is not empty:

Remove the front node

If the node is Goal:

Return the reconstructed path

For each neighbor in Up, Down, Left, Right:

If inside the grid, not blocked,

and not visited:

Mark visited

Store its parent

Add it to the queue

Return failure

2.3 Manhattan Distance Heuristic

Manhattan distance estimates the minimum number of moves between the current cell and the goal.

[
h(n)=|r_n-r_G|+|c_n-c_G|
]

For the start position ((1,1)) and goal ((5,5)):

[
h(S)=|1-5|+|1-5|=8
]

Therefore, at least eight moves are required. Standard BFS does not use this heuristic to order nodes; it explores them using FIFO order. Manhattan distance is commonly used to guide A* search.

2.4 Step-by-Step Node Expansion

The neighbor order is Up → Down → Left → Right. Obstacles, out-of-bounds cells, and visited cells are ignored.

Step	Expanded node	Newly added nodes
1	(1,1)	(2,1), (1,2)
2	(2,1)	(3,1)
3	(1,2)	(1,3)
4	(3,1)	(4,1), (3,2)
5	(1,3)	(1,4)
6	(4,1)	(5,1), (4,2)
7	(3,2)	(3,3)
8	(1,4)	(2,4), (1,5)
9	(5,1)	(5,2)
10	(4,2)	None
11	(3,3)	(3,4)
12	(2,4)	(2,5)
13	(1,5)	None
14	(5,2)	(5,3)
15	(3,4)	(3,5)
16	(2,5)	None
17	(5,3)	(5,4)
18	(3,5)	(4,5)
19	(5,4)	(5,5)
20	(4,5)	None

21	(5,5)	Goal reached

The goal is reached at Step 21. Parent pointers recorded during node discovery are used to reconstruct the shortest path.

2.5 Optimal Path and Cost

The shortest path found by BFS is:

$$
\begin{aligned}
(1,1)&\rightarrow(2,1)\rightarrow(3,1)\\
&\rightarrow(4,1)\rightarrow(5,1)\rightarrow(5,2)\\
&\rightarrow(5,3)\rightarrow(5,4)\rightarrow(5,5)
\end{aligned}
$$

The robot moves down the first column and then right along the bottom row, avoiding all obstacles.

Since each move costs 1 and the path contains eight moves:

$$
\text{Total Cost}=8\times1=\boxed{8}
$$

Therefore, the optimal path cost is **8 units**.

2.6 Complexity Analysis

Let (V) represent the number of traversable cells and (E) the number of valid connections between neighboring cells.

- **Time complexity:** (O(V+E))
- **Space complexity:** (O(V))

For an (R \times C) grid, both complexities are (O(RC)), because each cell has at most four neighbors.

Conclusion

BFS finds the shortest path from the start to the goal while avoiding the specified obstacles. For the illustrative grid, the robot reaches the goal in eight moves with a total cost of 8 units. The FIFO queue, visited-node tracking, and parent pointers ensure systematic exploration and path reconstruction.

QUESTION 3: Autonomous Rescue Robot Using Intelligent Agents and Online Search

3.1 Introduction

An autonomous rescue robot must reach a survivor in a disaster-affected building. The environment contains obstacles and risky zones, and the robot can observe only adjacent cells. Therefore, it must update its map, avoid blocked paths, and select a minimum-cost route.

Uniform Cost Search (UCS) combined with online search is suitable for this problem.

3.2 Constraint Analysis

Constraint	Description
Movement	Up, Down, Left, Right
Blocked cells	Cannot be entered
Normal movement cost	1
Risky-zone entry cost	3 (1 + 2 penalty)
Limited sensing	Only adjacent cells are observable
Dynamic updates	The robot updates its map after sensing
Objective	Reach the survivor with minimum cost

The total path cost is:

[
C(P)=\sum_{i=1}^{n}(1+2R_i)
]

Here, (R_i=1) when entering a risky zone and (R_i=0) otherwise.

For example, five normal moves and two risky-zone entries cost (5+2(3)=11).

3.3 Proposed Algorithm

Uniform Cost Search (UCS): UCS expands the position with the lowest accumulated path cost. It is suitable because normal and risky movements have different costs.

Online Search Agent: The robot senses nearby cells, updates its internal map, and changes its route when it discovers obstacles or risky zones.

Algorithm

1. Initialize the map and starting position S.

2. Observe adjacent cells.

3. Mark cells as free, blocked, or risky.

4. Assign costs:

   Normal cell = 1

   Risky cell = 3

   Blocked cell = prohibited

5. Apply UCS to the known map.

6. Move along the lowest-cost known route.

7. Update the map after each observation.

8. Recalculate the route when new information appears.

9. Repeat until the goal G is reached

or no feasible route remains.

3.4 Working Procedure

1. **Initialization:** The robot starts at S and creates an internal map.
2. **Sensing:** It identifies nearby free, blocked, and risky cells.
3. **Cost calculation:** It assigns a cost of 1 to normal moves and 3 to risky-zone entries.
4. **Route selection:** UCS selects the route with the lowest accumulated cost.
5. **Movement:** The robot moves to the next cell and observes its surroundings.
6. **Replanning:** If new obstacles or risks are discovered, the robot updates its map and recalculates the route.

3.5 Application of AI Concepts

Intelligent Agent

An intelligent agent perceives its environment and takes actions to achieve a goal.

- **Sensors:** Detect nearby obstacles and risky zones.
- **Internal state:** Stores the known map and current position.
- **Decision mechanism:** Uses UCS to select a route.
- **Actuators:** Execute movements.
- **Goal:** Reach the survivor safely with minimum cost.

Rationality

A rational robot chooses actions that help it reach the survivor, avoid blocked cells, minimize travel cost, and respond to new information. Because the environment is only partially observable, it must revise decisions as it learns more.

Environment Classification

Property	Classification
Observability	Partially observable
State space	Discrete
Actions	Discrete
Planning	Sequential
Knowledge	Initially incomplete
Movement model	Deterministic, assuming reliable actions

3.6 Cost Comparison

Route	Normal Moves	Risky Entries	Total Cost
A	6	2	12
B	8	0	8

Although Route A is shorter, Route B has a lower total cost. Therefore, Route B is preferred if it is feasible and satisfies the safety requirements.

3.7 Justification for UCS and Online Search

1. UCS considers total path cost rather than only the number of moves.
2. It supports different movement costs.
3. Online search allows the robot to explore without a complete map.
4. The robot can update its map and replan when new obstacles are found.
5. Known hazardous cells can be assigned higher costs or prohibited when necessary.

Limitation: UCS finds an optimal path when the relevant map is sufficiently known and movement costs are non-negative. With incomplete information, the robot cannot guarantee the globally optimal route before exploration.

Conclusion

The rescue robot demonstrates the use of intelligent agents, rational decision-making, and online search in a partially observable environment. UCS helps select minimum-cost routes,

while online exploration enables the robot to discover obstacles and update its map. Together, these methods support efficient and safer navigation toward the survivor.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understandin g	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided